

Subroutines and Control Abstraction

Exception Handling

Exception Handling

- Exception:
 - A hardware-detected run-time error or unusual condition detected by software.
- Examples
 - arithmetic overflow
 - end-of-file on input
 - wrong type for input data
 - divide by zero
 - user-defined conditions, not necessarily errors

Exception Handling

- Three important operations performed by exception handlers
 - compensate for exception to allow program to continue
 - “out of memory” may request more memory
 - if cannot be handled locally, handler may do local clean-up
 - e.g., call destructors
 - re-raises the exception to propagate upward
 - if recovery not possible, an error message can be printed

Handling the Exceptions

1. Invent an “error” value different from normal value. Return this value in case of an error.
 - Problem: It may not work in some cases, not applicable in general.
 - Ex. For a stack of integers, if we keep -1 as error value to be returned and top of stack is also set to -1, then it is not applicable.
2. Global variable
 - If error occurs, we set it to -1, else set it to 1.
 - After each call, we check value of error, if -1, then print an error message.
 - The library function sets the variable with a normal value if the call was successful.
 - The caller checks the error value after every call.

Handling the Exceptions Cont..

try{

push()

pop()

empty ()
}

catch push_err () {...}

catch pop_err () {...}

catch(...) { }
// for any kind of
exception-default case

paranthesis.c

init() { }

push() {

--

--

throw push_err
}

pop(){

--

--

throw pop_err
}

empty() { }

stack.c

- This facility is available as a part of language implementation itself.
- If error occurs in pop() inside, then the rest of the code is not executed.
- Control comes to 1st catch handler, check if the exception type matches, if found, then execute it else check for the next catch block.
- The transfer mechanism is implemented as a part of language itself.

Multiple handlers in C++

```
try {                // try to read from file
    ...
    // potentially complicated sequence of operations
    // involving many calls to stream I/O routines
    ...
} catch(end_of_file) {
    ...
} catch(io_error e) {
    // handler for any io_error other than end_of_file
    ...
} catch(...) {
    // handler for any exception not previously named
    // (in this case, the triple-dot ellipsis is a valid C++ token;
    // it does not indicate missing code)
}
```

Passing parameter along with Exception

- We can pass a parameter along with the exception.
- Ex. In case of Set data structure (No duplicates can be inserted)

```
Set_insert(int e){  
    ...  
    throw duplicateelement(e);  
}  
catch duplicateelement(x) { ...}
```

Checked Exception Example

- FileNotFoundException is a checked exception in Java.
- Try to read file with handle FileNotFoundException

```
public static void main(String[] args) {
```

```
    FileReader file = new FileReader("somefile.txt");}
```

compile time error with message – Unhandled exception type
FileNotFoundException.

```
public static void main(String[] args) {
```

```
    try{
```

```
        FileReader file = new FileReader("somefile.txt");}
```

```
    catch (FileNotFoundException e) {
```

```
        //Alternate logic
```

```
        e.printStackTrace(); }
```

```
}
```

Example of Unchecked Exceptions

NullPointerException.

```
public static void main(String[] args) {  
    try {  
        FileReader file = new FileReader("pom.xml");  
        file = null;  
        file.read();  
    }  
    catch (IOException e) {  
        //Alternate logic  
        e.printStackTrace(); }  
}
```

Implementation of Exception Handling

- Stack of Handlers
- The implicit (default) handler is created for entry function defined in a program.
- The default handler calls the Epilogue of the function and re-throws an exception that it has received.

Implementation of Exception Handling

- If `g ( )` throws an exception, therefore `g( )` can't continue normally.
- Checks its default handler that calls its epilogue which pops out its activation record (Top is updated).
- If the exception type is matched, then that handler is invoked.
- If the exception type is not matched, then its default handler is called which
 - pops out frame of `f()`
- This goes till for the default handler of `main()` and pops out the frame of `main()`
 - as well and language provides handler for that case which shows an appropriate message and terminate the program.

If the exception does not occur then we pop out frame of `f( )`

Compile Time Table of Handlers

Address of protected code	Address of corresponding handler
main()	Address of default handler of main()
f ()	Address of default handler of f ()
try_f()	Address of explicit handler of try_f()
g()	Address of default handler of g()

- This table or data structure is generated at compile time and is made available at run time.
- When an exception occurs in g () (throw e), search in the table with PC as the key to find the handler for corresponding block.
- In this case default handler of g() is invoked.
- Then, from the frame of default handler of g(), we get the return address of try_f() if the exception occurs else we get the return address of default handler of f() in case exception does not occur.

Multiple Catch Statements

```
try{ }  
catch e1 { }  
catch e2{ }  
...  
...  
catch en{ }
```

- How to construct compile time table in this case?
- We provide address of switch statement which will take the type of exception
and invoke corresponding handler.

Independent Compilation of Files

- What if the code fragments / functions are compiled independently?
- Example:
- `main( )` & `f ( )` in source file 1
- `g( )` in source file 2
- A hybrid approach:
- The compiler creates a separate table for each subroutine, and
- Each stack frame contains a pointer to the appropriate table.
- At the time of linking, linker will stick or glue these subtables together and finally sorts them in an ascending order to enable binary search